Formalisation d'un sous-langage d'eBPF à l'aide d'un assistant à la preuve.

AROCENA Thomas
Mines Nancy

dernière modification : 08/06/2025

Tuteurs : Ghilain BERGERON, Stephan MERZ, Vincent TRÉLAT

Résumé : Ce travail s'inscrit dans le cadre d'un projet de recherche réalisé durant ma formation d'ingénieur aux Mines de Nancy, en collaboration avec l'équipe VERIDIS du LORIA. Il porte sur la formalisation d'un sous-ensemble du langage eBPF (extended Berkeley Packet Filter) à l'aide de l'assistant à la preuve Lean . eBPF est un langage permettant d'exécuter du code dans le noyau Linux, offrant des possibilités avancées, par exemple pour le filtrage réseau, mais exigeant de fortes garanties de sûreté et de terminaison, assurées notamment par un vérificateur intégré.
L'objectif du projet est de poser les bases d'un compilateur certifié pour eBPF , en commençant par la formalisation d'un sous-langage restreint. Ce dernier comprend des instructions élémentaires : affectations, opérations arithmétiques et binaires, sauts conditionnels et appels de fonctions. Une syntaxe formelle a été définie dans Lean , ainsi qu'une sémantique concrète fondée sur les états du système.
La deuxième partie du travail introduit une interprétation abstraite du langage, inspirée des techniques utilisées dans le vérificateur du noyau Linux. Deux représentations abstraites ont été développées : les tristates, qui modélisent l'incertitude bit à bit, et les intervalles, qui encadrent les valeurs possibles des registres. Une série de propriétés et d'opérateurs ont été formalisés sur ces domaines abstraits, avec une attention particulière portée à la gestion des dépassements liés au type `UInt64`.
L'ensemble permet de définir une sémantique abstraite du langage, plus adaptée à l'analyse statique que la sémantique concrète, et ouvrant la voie à la vérification automatique de propriétés comme la terminaison ou l'absence de comportements indéfinis. Ce travail constitue une première étape vers la certification complète de programmes eBPF .

Table des matières

1 Introduction **2**
1.1 Notations . . . 2
1.2 Présentation d'eBPF . . . 2
1.2.1 Motivation . . . 2
1.2.2 Le langage eBPF . . . 3
1.2.3 Le vérificateur d'eBPF . . . 3
1.3 Présentation de Lean . . . 3

2 Formalisation **4**
2.1 Formalisation de la syntaxe . . . 4
2.1.1 Représentation de la mémoire . . . 5
2.1.2 Représentation d'un programme . . . 5
2.2 Sémantique . . . 6
2.2.1 Représentation d'un état du système . . . 6
2.2.2 Transitions d'états . . . 6
2.3 Exemple . . . 6

3 Interprétation abstraite **7**
3.1 Motivation . . . 7
3.2 Les entiers 64 bits et le type `UInt64` . . . 7
3.2.1 Définition . . . 7
3.2.2 Théorèmes . . . 8
3.3 Les tristates et le type `Tnum` . . . 9
3.3.1 Définition . . . 10
3.3.2 Opérations binaires . . . 10
3.3.3 Opérations arithmétiques . . . 13
3.4 Les intervalles d'entiers 64 bits et le type `I` . . . 14
3.4.1 Définition . . . 14
3.4.2 Opérations arithmétiques . . . 15
3.4.3 Opérations croisées entre `Tnum` et `I` . . . 16
3.5 Sémantique abstraite . . . 18
3.5.1 Ensemble des états par point de programme . . . 18
3.5.2 Initialisation . . . 18
3.5.3 Étape de propagation (récurrence) . . . 18
3.5.4 Esquisse d'implémentation . . . 19

4 Conclusion **20**
4.1 Travail accompli . . . 20
4.2 Perspectives . . . 20
4.3 Expérience personnelle . . . 20

Annexes **22**

A Codes Lean **22**

1 Introduction

1.1 Notations

Dans ce rapport, les objets seront notés en `typewriter` lorsqu'ils sont considérés comme du code, et en *italique* lorsqu'on les manipule comme objets mathématiques. Le projet consistant à naviguer entre la représentation formelle et l'implémentation concrète, beaucoup d'objets apparaîtront sous les deux formes. Cette distinction vise à faciliter le passage d'un monde à l'autre, bien qu'il s'agisse en réalité d'objets différents.

Par exemple, on peut considérer les entiers 4 et 5 et l'addition $4+5$ en tant qu'objets mathématiques, ou bien les objets `4`, `5` et `4 + 5` qui représentent des lignes de code, n'ayant a priori qu'une valeur syntaxique.

1.2 Présentation d'eBPF

1.2.1 Motivation

eBPF (extended Berkeley Packet Filter) est un langage de programmation pouvant être exécuté au niveau du noyau du système d'exploitation Linux. Le noyau est le logiciel chargé de gérer la communication entre le matériel et les logiciels. Il est particulièrement sensible car il s'exécute avant tout autre logiciel, et n'est généralement pas accessible directement par l'utilisateur. D'un autre côté, il peut être très utile de pouvoir exécuter du code dans le noyau. Par exemple, pour filtrer des paquets réseau, on peut soit recevoir tous les paquets et les transférer à un logiciel utilisateur qui les filtrera, soit exécuter un programme directement dans le noyau, qui effectuera le filtrage avant même de transférer les paquets. Il est facile de comprendre que, lorsque peu de paquets doivent être conservés, la seconde option réduit considérablement le nombre de requêtes et améliore les performances. Historiquement, le BPF (Berkeley Packet Filter) a été conçu dans ce but. On s'est ensuite aperçu que ce type de programmes pouvait être utile dans d'autres contextes, ce qui a conduit au développement du langage enrichi eBPF .

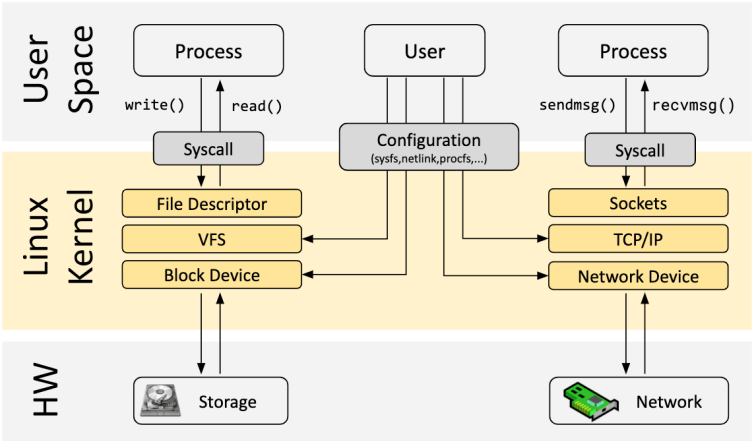

FIGURE 1 – Architecture du noyau Linux

1.2.2 Le langage eBPF

Il existe plusieurs manières d'écrire des programmes eBPF . Les plus courantes consistent à utiliser des bibliothèques adaptées aux langages C, C++ ou Python, qui permettent de traduire un code source écrit dans ces langages vers un code binaire spécifique à eBPF . Ce code est ensuite transféré au noyau, dans lequel il est vérifié, compilé, puis rattaché à une portion du noyau où il pourra être exécuté.

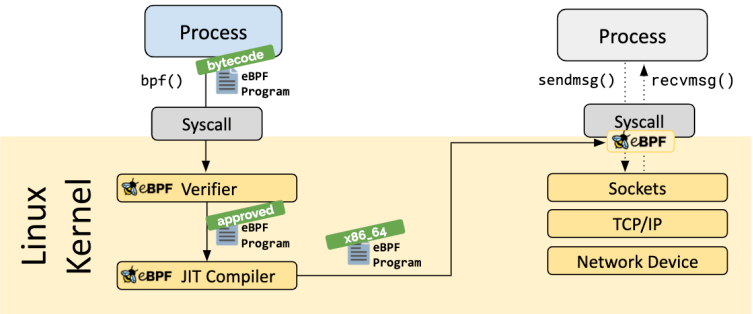

FIGURE 2 – Implémentation d'un programme eBPF

1.2.3 Le vérificateur d'eBPF

Puisque les programmes eBPF s'exécutent dans le noyau, il est crucial de s'assurer qu'ils ne peuvent ni boucler indéfiniment, ni contenir de code malveillant. C'est le rôle du vérificateur : un programme chargé d'analyser tout code eBPF avant son exécution. Le vérificateur prend un programme en entrée et détermine s'il est sûr. Si le programme est accepté, cela signifie qu'il termine, ne contient pas de comportements dangereux, et respecte les contraintes du noyau. Dans le cas contraire, le programme est rejeté.

1.3 Présentation de Lean

Lean est un langage de programmation fonctionnel et un assistant à la preuve. Il permet de définir des types, d'écrire des fonctions, de formaliser des théorèmes et de construire des preuves dont la validité est analysée par l'assistant à la preuve.

Par exemple, voici la définition du type des entiers naturels ℕ dans Lean :

```
inductive ℕ : Type
  | zero : ℕ
  | successeur (n : ℕ) : ℕ
```

La commande `inductive` permet de définir un type inductif. Ici, on introduit deux constructeurs : `zero`, représentant 0, et `successeur`, qui construit un entier à partir d'un autre. Ainsi, on peut écrire $1 =$ `successeur(zero)`, $2 =$ `successeur(successeur(zero))`, etc.

On peut alors définir l'addition de manière récursive :

```
def add : ℕ → ℕ → ℕ
  | m  zero     := m
  | m  (succ n) := succ (add m n)
```

Le symbole `|` introduit une définition par cas : si le second argument est `zero`, alors la somme est `m`. Sinon, si c'est `succ(n)`, on ajoute récursivement puis on applique le successeur.

Par exemple : `3 + 1 = 3 + successeur(zero) = successeur(3 + zero) = successeur(3) = 4`

Lean permet aussi de formaliser des propriétés et d'en démontrer les preuves, comme ci-dessous :

```
theorem zero_add (n : Nat) : 0 + n = n :=
  Nat.recOn (motive := fun x => 0 + x = x)
  n
  (show 0 + 0 = 0 from rfl)
  (fun (n : Nat) (ih : 0 + n = n) =>
  show 0 + succ n = succ n from
  calc 0 + succ n
  _ = succ (0 + n) := rfl
  _ = succ n       := by rw [ih])
```

Le théorème suivant démontre, par récurrence sur `n`, l'égalité élémentaire :

$$\forall n \in \mathbb{N},\ 0 + n = n$$

La preuve utilise le principe de récurrence fourni par Lean avec `Nat.recOn`.

- **Initialisation** : on montre que `0 + 0 = 0`, ce qui est immédiat par réflexivité (`rfl`).
- **Récurrence** : en supposant que l'égalité `0 + n = n` est vraie (hypothèse de récurrence `ih`), on souhaite prouver que :

  `0 + succ n = succ n`

  Pour cela, on utilise un raisonnement par calcul (`calc`) :

`0 + succ n = succ(0 + n)`	(définition de l'addition)
`= succ n`	(par hypothèse d'induction `ih`)

Cette preuve illustre la manière dont Lean permet de construire des raisonnements rigoureux pas à pas à l'aide de ce qu'on appelle des tactiques (`recOn`, `calc`, `rw`). Le théorème `zero_add` pourra lui aussi ensuite être réutilisé dans d'autres preuves par la suite. Pour aller plus loin, la documentation de Lean présente en détail l'ensemble des tactiques natives disponibles.[1](*Theorem Proving in Lean 4*).

2 Formalisation

Dans le cadre d'un projet plus large visant à construire un compilateur certifié pour eBPF, c'est-à-dire formellement vérifié à l'aide d'un assistant à la preuve, il est essentiel de commencer par une formalisation rigoureuse du langage. Dans ce parcours de recherche, je me suis concentré sur un sous-ensemble réduit d'instructions de eBPF. Nous allons ici formaliser des programmes n'utilisant que les instructions suivantes : affectation, addition, multiplication, opérations binaires OU et ET, sauts (inconditionnels et conditionnels par égalité), appels de fonction, et enfin l'instruction de sortie. Cette formalisation est bien sûr extensible à d'autres instructions dans un second temps. Formaliser un programme eBPF signifie ici construire une représentation syntaxique fidèle du langage ainsi qu'une sémantique décrivant l'effet de chaque instruction sur l'état de la machine.

2.1 Formalisation de la syntaxe

La formalisation syntaxique consiste à modéliser la structure des programmes : les objets manipulés, les instructions, les registres, etc. L'objectif est de construire une syntaxe en Lean aussi proche que possible de celle de eBPF, de façon à ce qu'un programme écrit en eBPF et sa formalisation soient facilement comparables.

2.1.1 Représentation de la mémoire

La mémoire dans eBPF repose sur 11 registres nommés R_0 à R_{10}, chacun capable de stocker un entier 64 bits. On définit donc un type `Register` pour représenter ces registres, et on utilise le type natif de LEAN pour les entiers 64 bits.

```
inductive Register
  | r0 | r1 | r2 | r3 | r4 | r5 | r6 | r7 | r8 | r9 | r10

abbrev Val := UInt64
```

2.1.2 Représentation d'un programme

Les instructions d'eBPF sont codées sur 64 bits, organisés selon le format suivant :

8 bits	4 bits	4 bits	16 bits	32 bits
Opcode	Dst Reg	Src Reg	Offset	Immediate

Chaque instruction d'eBPF est donc identifiée par un opcode, et paramétrée par des registres (source et destination), un offset, et une valeur immédiate. Certains de ces champs peuvent être inutilisés selon l'instruction. Par exemple, l'instruction `ADD r0, r1` s'interprète comme suit :

ADD|X|ALU	r_0	r_1	0	0

Dans notre formalisation LEAN , plutôt que de définir un type générique pour toutes les instructions et fournir des arguments parfois inutiles, nous préférons utiliser une approche plus structurée : chaque nom d'instruction est représenté par un constructeur du type `Statement`, avec uniquement les paramètres pertinents.

Instruction	Registre destination	Registre source	Offset	Valeur immédiate
MOV	✓	✓/✗	✗	✗/✓
ADD	✓	✓/✗	✗	✗/✓
MUL	✓	✓/✗	✗	✗/✓
OR	✓	✓/✗	✗	✗/✓
AND	✓	✓/✗	✗	✗/✓
JA	✗	✗	✓	✗
JEQ	✓	✓/✗	✓	✗/✓

Le symbole ✓/✗ signifie que l'argument peut être soit un registre, soit une valeur immédiate. On introduit donc un type `Argument` représentant cette alternative.

```
inductive Argument
  | reg : Register → Argument
  | imm : Val → Argument

inductive Statement
  | add : Register → Argument → Statement
  | or : Register → Argument → Statement
  | mov : Register → Argument → Statement
  | ja : Int → Statement
  | jeq : Register → Argument → Int → Statement
  | call : Int → Statement
  | exit : Statement
```

Un programme est alors simplement une liste d'instructions :

```
abbrev Program := List Statement
```

2.2 Sémantique

2.2.1 Représentation d'un état du système

L'état d'un programme à un instant donné se résume à deux informations :
— un compteur de programme (`pc`) indiquant l'instruction en cours ;
— une fonction donnant la valeur contenue dans chaque registre.
Cela se modélise naturellement par une structure :

```
structure State where
  pc : Int
  reg : Register → Val
```

2.2.2 Transitions d'états

La sémantique opérationnelle décrit comment l'on passe d'un état au suivant au cours de l'exécution. Pour chaque instruction, on définit donc une fonction prenant un état courant et produisant le nouvel état. C'est cette dynamique qui forme la relation binaire de transition entre états, notée $\mathcal{R}$. Autrement dit, $s\mathcal{R}s'$ signifie qu'en exécutant le programme à partir de l'état s, on obtient l'état s'. Cette relation dépend du programme exécuté : on utilise le compteur `pc` de s pour identifier l'instruction en cours dans la liste. Commençons par une fonction utilitaire pour modifier la valeur d'un registre :

```
def update (g : Register → Val) (α : Register) (v : Val) : Register → Val :=
  fun r => if r = α then v else g r
```

Voici maintenant la définition de l'instruction `add` :

```
def add (dst : Register) (src : Argument) (s : State) : State :=
  match src with
  | Argument.imm v_src => {
     pc := s.pc + 1,
     reg := update s.reg dst (s.reg dst + v_src)
  }
  | Argument.reg r_src => {
     pc := s.pc + 1,
     reg := update s.reg dst (s.reg dst + s.reg r_src)
  }
```

On définit de façon similaire une fonction pour chaque instruction du langage (voir annexe A.1). Une fois ces fonctions définies, on peut alors construire la relation $\mathcal{R}$, en regardant à chaque étape, selon le `pc`, quelle instruction appliquer à l'état courant. Par exemple, si `s.pc = 3` et que l'instruction à l'indice 3 du programme est `Statement.add Register.r0 (Argument.reg r1)`, alors on a :

$$s\mathcal{R}(\texttt{Statement.add Register.r0 (Argument.reg r1)})$$

En répétant ce processus à partir d'un état initial s_0, on génère la séquence complète d'états visités lors de l'exécution du programme.

2.3 Exemple

Exécution d'un exemple

Prenons pour illustrer cette sémantique un mini-programme eBPF effectuant une simple addition :

```
R1 ← 10
R2 ← 32
R0 ← R0 + R1
R0 ← R0 + R2
exit
```

Ce programme, formalisé sous forme de liste d'instructions dans LEAN , s'écrit :

```
def example_program : Program :=
  [ Statement.mov Register.r1 (Argument.imm 10),
    Statement.mov Register.r2 (Argument.imm 32),
    Statement.add Register.r0 (Argument.reg r1),
    Statement.add Register.r0 (Argument.reg r2),
    Statement.exit ]
```

En partant de l'état initial où tous les registres valent 0, et le compteur `pc` est à 0, l'exécution produit la séquence d'états suivante :

pc	Instruction	Registres après exécution
0	`Statement.mov Register.r1 (Argument.imm 10)`	r1 = 10
1	`Statement.mov Register.r2 (Argument.imm 32)`	r2 = 32
2	`Statement.add Register.r0 (Argument.reg r1)`	r0 = 10
3	`Statement.add Register.r0 (Argument.reg r2)`	r0 = 42
4	`Statement.exit`	(fin d'exécution)

À la fin du programme, la valeur contenue dans `r0` est donc `42`, résultat de l'addition des deux constantes.

Cet exemple illustre le fonctionnement de la sémantique opérationnelle définie précédemment et montre comment, à partir d'une suite d'instructions et d'un état initial, on peut dérouler mécaniquement l'exécution d'un programme.

3 Interprétation abstraite

3.1 Motivation

Nous avons formalisé un sous-langage d'eBPF, ce qui permet d'observer l'exécution d'un programme à partir d'un état initial donné. Cependant, il serait souhaitable d'obtenir des garanties plus générales sur un programme, telles que des propriétés de terminaison ou de correction, indépendamment de l'état initial, à l'image de ce que fait le vérificateur d'eBPF. Or, il n'est pas envisageable d'examiner toutes les exécutions possibles à partir de tous les états initiaux concrets. C'est ici qu'intervient l'**interprétation abstraite**. Elle consiste à approximer un ensemble d'états concrets par un **état abstrait**, et à définir une nouvelle sémantique, dite abstraite, qui décrit comment l'on passe d'un état abstrait au suivant.

Puisqu'une approximation abstraite introduit une perte d'information, toute propriété démontrée par la sémantique abstraite reste valide dans le monde concret. En revanche, la réciproque est fausse : certaines propriétés peuvent être vraies concrètement mais ne pas être vérifiables via l'abstraction. Cela signifie qu'en cas de doute, l'interprétation abstraite peut être trop prudente, mais jamais trop optimiste.

Dans la suite, l'adjectif *abstrait* se rapportera à tout ce qui concerne la sémantique abstraite, tandis que l'on qualifiera de *concret* tout ce qui relève de la sémantique définie en 2.2.

Comme vu dans la partie 2, les valeurs concrètes sont des entiers 64 bits, représentés dans $\sqsubseteq\exists\forall\mathbb{N}$ par le type `UInt64`. Les domaines abstraits que nous allons considérer sont les **tristates** et les **intervalles** d'entiers 64 bits, introduits respectivement dans les parties 3.3 et 3.4.

Le choix d'un domaine abstrait est un point crucial : il s'agit de trouver un équilibre entre une approximation suffisamment fine pour capturer des propriétés intéressantes, et suffisamment simple pour permettre une sémantique abstraite calculable. Toute une théorie encadre ces questions, voir par exemple [5] (Cousot, 2001). Nous ne détaillerons pas ici ce choix, car nous réutilisons les domaines employés dans le vérificateur d'eBPF. Des justifications formelles de la pertinence des tristates sont disponibles dans [6] (Vishwanathan et al., 2022).

3.2 Les entiers 64 bits et le type `UInt64`

3.2.1 Définition

Les valeurs contenues dans les registres sont des entiers 64 bits. Pour les représenter, il est naturel d'utiliser le type `UInt64` (pour unsigned 64-bit integer) de $\sqsubseteq\exists\forall\mathbb{N}$, défini comme un vecteur binaire de taille 64, aussi interprétable comme un entier naturel modulo 2^{64}. On dispose de fonctions de conversion entre `UInt64`, `BitVec` et $\mathbb{N}$.

Pour expliciter le type d'un élément, on peut écrire la valeur entre parenthèses, suivie de `:` et du type. La commande `#check` permet d'afficher le type d'un objet, tandis que `#eval` en donne la valeur.

```
#check 5                      /- 5 : ℕ -/
#check (5 : UInt64)           /- 5 : UInt64 -/
#check (5 : ℕ).toUInt64       /- 5 : UInt64 -/
#check (5 : UInt64).toNat     /- 5 : ℕ -/
#check (5 : UInt64).toBitVec  /- 5#64 : BitVec -/
```

```
#eval 5                        /- 5 -/
#eval (5 : UInt64)             /- 5 -/
#eval (2^64 : Nat).toUInt64    /- 0 -/
```

La conversion `.toNat` transforme un `UInt64` en entier naturel. Réciproquement, `.toUInt64` convertit un $\mathbb{N}$ en `UInt64`, avec une réduction modulo 2^{64}, comme on le voit dans le dernier exemple. Enfin, `.toBitVec` transforme un `UInt64` en vecteur binaire.

Un inconvénient notable du type `UInt64` est qu'il est peu utilisé dans la bibliothèque `MathLib` qui est la bibliothèque mathématique principale de ∃∀N , rassemblant un vaste ensemble de définitions et théorèmes. Peu de théorèmes sur les `UInt64` y sont directement disponibles, bien qu'il soit souvent possible d'en dériver depuis ceux sur `BitVec` ou $\mathbb{N}$.

3.2.2 Théorèmes

Pour manipuler des intervalles, nous utilisons le type `Interval` de `Mathlib`, qui nécessite un ordre sur le type considéré. Bien que `UInt64` possède une relation $\leq$, elle n'est pas directement intégrée comme ordre partiel. Il faut donc définir les instances suivantes :

```
instance : Preorder UInt64 where
   le_refl := by intros a; exact UInt64.le_refl a
   le_trans := by intros a b c hab hbc; exact UInt64.le_trans hab hbc
   lt := fun a b => a ≤ b ∧ ¬b ≤ a
   lt_iff_le_not_le := by intros a b; rfl

instance : PartialOrder UInt64 where
   le_antisymm := by intro a b hab; exact UInt64.le_antisymm hab
```

L'instruction `instance` permet d'associer à un type des propriétés usuelles (ordre, égalité, etc.), comme on le fait ici.

Un exemple de propriété utile sur les intervalles est la suivante :

$$(a_1 \leq a_2) \wedge (b_1 \leq b_2) \Rightarrow a_1 + b_1 \leq a_2 + b_2$$

Cette propriété est vraie sur $\mathbb{N}$, mais pas toujours sur `UInt64`, car l'addition est effectuée modulo 2^{64}. Ainsi, si $a_2 + b_2$ dépasse cette borne, le résultat peut revenir à zéro et inverser l'inégalité. Il faut donc ajouter une hypothèse de borne :

```
theorem add_le_add (p : a ≤ b) (q : c ≤ d) (h : b.toNat + d.toNat ≤ UInt64.size-1) : a + c ≤
    b + d := by
   apply toNat_add_le_umax at h
   rw [UInt64.le_iff_toNat_le] at p q ⊢
   have h1 := Nat.add_le_add p q
   have h2 : (a + c).toNat ≤ a.toNat + c.toNat := by exact toNat_ge_add
   trans
   · exact h2
   · rw [h] at h1 ; exact h1
```

Les lemmes `toNat_add_le_umax` et `toNat_ge_add` assurent que l'addition sur `UInt64` est cohérente avec celle sur $\mathbb{N}$ tant qu'on reste sous la borne 2^{64} (cf. annexe A.2).

On souhaite également disposer des propriétés :

$$\min(a, b) \leq a \qquad \text{et} \qquad a \leq \max(a, b)$$

On les démontre en montrant que min et max sur `UInt64` coïncident avec ceux sur $\mathbb{N}$ via `toNat` :

```
theorem min_toNat_eq : (min a b).toNat = min a.toNat b.toNat := by
   change (if a ≤ b then a else b).toNat = if a.toNat ≤ b.toNat then a.toNat else b.toNat
   split <;> split <;> trivial

theorem min_le_left : min a b ≤ a := by
   change (min a b).toNat ≤ a.toNat
   rw [UInt64.min_toNat_eq]
   apply Nat.min_le_left
```

Un raisonnement similaire permet d'établir les propriétés sur max (voir annexe A.3).

$\wedge$ **(ET bit à bit)** Comme nous le verrons dans la section 3.3 sur les tristates, on utilise des opérations binaires bit à bit. En Lean, l'opérateur `&&&` correspond au *ET bit à bit*, défini comme :

$\wedge$	0	1
0	0	0
1	0	1

Le complément bit à bit $\neg$ est noté `~~~` dans ∃∀N et est défini ainsi : $\neg 0 = 1$, $\neg 1 = 0$. On démontre alors les théorèmes suivants :

```
variable {x y z : UInt64}

theorem and_zero : x &&& 0 = 0
theorem zero_and : 0 &&& x = 0
theorem and_not_eq_zero : x &&& (~~~ x) = 0
theorem and_comm : x &&& y = y &&& x
theorem and_assoc : x &&& y &&& z = x &&& (y &&& z)
```

Ces propriétés dérivent naturellement de celles sur les vecteurs binaires.

Décalages : On définit également les décalages à droite (*Rshift*) et à gauche (*Lshift*) sur `UInt64`, qui insèrent des zéros aux extrémités :

$$\begin{array}{rcccc} \mathit{Rshift} & : & \mathit{UInt64} & \to & \mathit{UInt64} \\ & & (b_{64}b_{63}\dots b_0) & \mapsto & (0b_{64}b_{63}\dots b_1) \end{array}$$

$$\begin{array}{rcccc} \mathit{Lshift} & : & \mathit{UInt64} & \to & \mathit{UInt64} \\ & & (b_{64}b_{63}\dots b_0) & \mapsto & (b_{63}\dots b_0 0) \end{array}$$

On peut décaler plusieurs fois en composant par le même décalage. Ces opérations de décalage n-fois sont notées en Lean `a <<< n` et `a >>> n` pour `n`$\in \mathbb{N}$. On peut les définir ainsi sur les `UInt64` :

```
protected def Lshift (x : UInt64) (s : Nat) : UInt64 := UInt64.ofNat (x.toNat <<< s)
instance : HShiftLeft (UInt64) Nat (UInt64) := ⟨.Lshift⟩

protected def Rshift (x : UInt64) (s : Nat) : UInt64 := UInt64.ofNat (x.toNat >>> s)
instance : HShiftRight (UInt64) Nat (UInt64) := ⟨.Rshift⟩
```

On peut alors démontrer plusieurs propriétés intéressantes, en particulier la bonne interaction entre décalage et opérations binaires :

```
theorem zero_lshift {k : Nat} : (0 : UInt64) <<< k = 0
theorem lshift_and_comm {k : Nat} : (a <<< k) &&& (b <<< k) = (a &&& b) <<< k
theorem lshift_wf (h : a &&& b = 0) : (a <<< k) &&& (b <<< k) = 0 := by
   have : (a <<< k) &&& (b <<< k) = (a &&& b) <<< k := by exact lshift_and_comm
   rw [h] at this
   rw [this]
   exact zero_lshift

theorem zero_rshift {k : Nat} : (0 : UInt64) >>> k = 0
theorem rshift_and_comm {k : Nat} : (a >>> k) &&& (b >>> k) = (a &&& b) >>> k
theorem rshift_wf (h : a &&& b = 0) : (a >>> k) &&& (b >>> k) = 0 := by
   have : (a >>> k) &&& (b >>> k) = (a &&& b) >>> k := by admit
   rw [h] at this
   rw [this]
   exact sorry
```

3.3 Les tristates et le type `Tnum`

Dans l'interprétation abstraite on considère donc des *tristates* qui représentent l'information que l'on a sur les bits. Il n'y a pas de théorie sur les tristates sur ∃∀N et il faut donc la créer de toutes pièces.

3.3.1 Définition

Une manière intuitive de représenter les tristates est d'utiliser la notation suivante : ? lorsqu'on ne connait pas un bit et la valeur correspondante sinon. Par exemple : 00 ?11 ?0 $= \{0001100, 0001110, 0011100, 0011110\}$. Cependant, cette représentation est peu pratique pour effectuer des opérations : elle masque en réalité une écriture ternaire plutôt que binaire. On préfère donc représenter un tristate par un couple de nombres binaires :

— une **valeur** qui indique les bits connus (et contient des zéros là où les bits sont inconnus),
— un **masque** qui indique, avec des 1, les positions des inconnues.

Par exemple : 00 ?11 ?0 se représente par :

$$\text{valeur} = 0001100, \quad \text{masque} = 0010010$$

Un tristate est donc un couple de binaires (v, m) qui satisfait la propriété :

$$v \wedge m = 0$$

Autrement dit, un bit ne peut pas être à la fois certain (dans v) et incertain (dans m). On définit ainsi le type `Tnum` (pour *tristate number*) dans ∃∀N :

```
structure Tnum where
   v : UInt64
   m : UInt64
   wf : v &&& m = 0
```

où `v` est la valeur, `m` est le masque et `wf`(pour well formed) est une preuve de `v &&& m` $= 0$ On dit qu'une valeur concrète `u` $\in$ `UInt64` appartient à un tristate `t = (v, m)` $\in$ `Tnum` si tous les bits certains de `t` correspondent à ceux de `u`.

Pour gérer les opérations entre un tristate et une valeur immédiate on veut pourvoir plonger les `UInt64` dans les `Tnum`. Une constante est un tristate sans incertitude. On peut donc définir la fonction suivante pour injecter un `UInt64` dans un `Tnum` :

```
def const (a : UInt64) : Tnum where
   v := a
   m := 0
   wf := by UInt64.and_zero
```

On vérifie bien que `const(a)` est un tristate bien formé : la propriété `a &&& 0 = 0` a été démontrée précédemment (voir `UInt64.and_zero`). La tactique `by` applique directement ce lemme.

3.3.2 Opérations binaires

Nous allons étendre les opérations définies sur `UInt64` aux tristates, c'est-à-dire aux éléments de type `Tnum`. Pour commencer, rappelons que, sur les `UInt64`, nous avons défini les décalages à gauche et à droite (notés respectivement `Lshift` et `Rshift`). Nous étendons naturellement ces opérations aux `Tnum` de la manière suivante :

$$\begin{array}{rccc} \mathit{Rshift}: & \mathit{Tnum} & \to & \mathit{Tnum} \\ & (v,m) & \mapsto & (\mathit{Rshift}(v), \mathit{Rshift}(m)) \end{array} \quad \text{et} \quad \begin{array}{rccc} \mathit{Lshift}_n: & \mathit{Tnum} & \to & \mathit{Tnum} \\ & (v,m) & \mapsto & (\mathit{Lshift}(v), \mathit{Lshift}(m)) \end{array}$$

On peut ensuite étendre la notation `<<< n` et `>>> n` à `Tnum`. En ∃∀N nous pouvons définir :

```
def rshift (t : Tnum) (k : Nat) : Tnum where
   v := t.v >>> k.toUInt64,
   m := t.m >>> k.toUInt64,
   wf := by apply UInt64.rshift_wf; exact t.wf

instance : HShiftRight Tnum Nat Tnum := ⟨rshift⟩

def lshift (t : Tnum) (k : Nat) : Tnum where
   v := t.v <<< k.toUInt64,
   m := t.m <<< k.toUInt64,
   wf := by apply UInt64.lshift_wf; exact t.wf

   instance : HShiftLeft Tnum Nat Tnum := ⟨lshift⟩
```

L'ensemble de ces définitions garantit que le résultat reste un élément de `Tnum` (c'est-à-dire, la propriété de validité `v &&& m = 0` est préservée).

Nous souhaitons ensuite définir les opérations binaires $\wedge$, $\vee$ et $\oplus$ sur les `Tnum`. Pour ce faire, l'idée est de partir d'une réflexion bit à bit sur les opérations classiques sur `UInt64` et d'en tirer la définition pour les tristates en tenant compte du masque d'inconnues.

$\wedge$ (ET bit à bit) : Une approche intuitive consiste à propager l'inconnue bit à bit :

$$\begin{array}{r} 0 \\ \wedge\ 0 \\ \hline =\ 0 \end{array} \quad \begin{array}{r} 1 \\ \wedge\ 0 \\ \hline =\ 0 \end{array} \quad \begin{array}{r} 0 \\ \wedge\ 1 \\ \hline =\ 0 \end{array} \quad \begin{array}{r} 1 \\ \wedge\ 1 \\ \hline =\ 1 \end{array} \quad \begin{array}{r} ? \\ \wedge\ 1 \\ \hline =\ ? \end{array} \quad \begin{array}{r} 1 \\ \wedge\ ? \\ \hline =\ ? \end{array} \quad \begin{array}{r} ? \\ \wedge\ 0 \\ \hline =\ 0 \end{array} \quad \begin{array}{r} 0 \\ \wedge\ ? \\ \hline =\ 0 \end{array} \quad \begin{array}{r} ? \\ \wedge\ ? \\ \hline =\ ? \end{array}$$

On pourrait faire les opérations ainsi bit à bit mais ce n'est pas optimal. Pour formuler une définition efficace sur `Tnum`, on procède comme suit :

Soient $t_1 = (v_1, m_1)$ et $t_2 = (v_2, m_2)$. D'une part, la partie certaine (la *valeur*) de $t_1 \wedge t_2$ doit être donnée par :

$$v = v_1 \wedge v_2.$$

En effet dans la valeur, ? est remplacé par 0 ce qui, si l'on regarde les opérations ci-dessus, correspond exactement à l'opérateur $\wedge$ usuel. Pour propager l'inconnue, on définit pour chaque tristate une nouvelle valeur en considérant l'union(OU) de la valeur et du masque :

$$\alpha = v_1 \vee m_1 \quad \text{et} \quad \beta = v_2 \vee m_2.$$

Cela signifie que α (resp. β) contient un 1 à une position si le bit correspondant de t_1 (resp. t_2) est soit connu à 1, soit inconnu. Ensuite, pour éliminer le cas où deux bits connus à 1 se rencontrent (car dans ce cas, le masque ne doit pas signaler d'incertitude), on utilise $\neg v$. Au final, le masque du résultat est défini par :

$$m = \alpha \wedge \beta \wedge (\neg v).$$

En effet, $\neg v$ possède des 0 uniquement aux position où l'on a un 1 connus simultanément pour t_1 et pour t_2. Ainsi, la définition de $\wedge$ sur les tristates s'exprime par :

$$\begin{array}{rccc} \wedge : & \mathit{Tnum} \times \mathit{Tnum} & \to & \mathit{Tnum} \\ & (t_1, t_2) & \mapsto & \big(v_1 \wedge v_2,\ \alpha \wedge \beta \wedge (\neg v)\big) \end{array}$$

En ∃∀ℕ , cela s'implémente ainsi :

```
def and (t₁ t₂ : Tnum) : Tnum :=
   let α := t₁.v ||| t₁.m
   let β := t₂.v ||| t₂.m
   let val := t₁.v &&& t₂.v
   { v := val, m := α &&& β &&& (~~~val),
      wf := by
      have : α &&& β &&& (~~~val) = (~~~val) &&& α &&& β := by rw [UInt64.and_comm,
    UInt64.and_assoc],
      rw [this, ← UInt64.and_assoc, ← UInt64.and_assoc],
      rw [UInt64.and_not_eq_zero, UInt64.zero_and, UInt64.zero_and]
   }
```

La preuve dans le champ `wf` s'appuie sur la commutativité et l'associativité de $\wedge$ (ici symbolisé par `&&&`) ainsi que sur le théorème `UInt64.and_not_eq_zero` qui assure que `v &&& (~~~v) = 0`.

$\vee$ (OU bit à bit) : Pour l'opération $\vee$, on part également de la définition usuelle sur les bits :

$\vee$	0	1
0	0	1
1	1	1

La propagation de l'inconnue intuitive bit à bit se résume alors ainsi :

$$\begin{array}{r} ? \\ \vee\ 1 \\ \hline =\ 1 \end{array} \quad \begin{array}{r} 1 \\ \vee\ ? \\ \hline =\ 1 \end{array} \quad \begin{array}{r} ? \\ \vee\ 0 \\ \hline =\ ? \end{array} \quad \begin{array}{r} 0 \\ \vee\ ? \\ \hline =\ ? \end{array} \quad \begin{array}{r} ? \\ \wedge\ ? \\ \hline =\ ? \end{array}$$

Là encore il est préférable d'opter pour une méthode utilisant les valeurs et les masques.En considérant deux tristates $t_1 = (v_1, m_1)$ et $t_2 = (v_2, m_2)$, la partie certaine du résultat est donnée par :

$$v = v_1 \vee v_2.$$

En effet, si l'on regarde les opérations bit à bit en remplaçant ? par 0 on obtient l'opérateur $\vee$ usuel. Pour le masque, on souhaite signaler l'incertitude provenant de t_1 ou t_2. On définit ainsi :

$$\mu = m_1 \vee m_2,$$

et le masque final est obtenu en "éliminant" les positions où la valeur de t_1 ou celle de t_2 possède un 1 certain (c'est-à-dire, en appliquant $\neg v$). Le résultat est donc :

$$m = \mu \wedge (\neg v).$$

La définition formelle est :

$$\begin{array}{rccc} \vee : & Tnum \times Tnum & \to & Tnum \\ & (t_1, t_2) & \mapsto & \big(v,\ \mu \wedge (\neg v)\big) \end{array}$$

En ∃∀ℕ , on peut écrire :

```
def or (t₁ t₂ : Tnum) : Tnum :=
   let val := t₁.v ||| t₂.v
   let μ := t₁.m ||| t₂.m
   { v := val, m := μ &&& (~~~val),
      wf := by
      have : val &&& (μ &&& ~~~val) = ~~~val &&& val &&& μ := by rw [← UInt64.and_assoc,
    UInt64.and_comm, ← UInt64.and_assoc],
      have h : ~~~val &&& val = 0 := by rw [UInt64.and_comm, UInt64.and_not_eq_zero],
      rw [this, h, UInt64.zero_and]
   }
```

⊕ (XOR bit à bit) : Pour l'opérateur $\oplus$, la définition habituelle est la suivante :

$\oplus$	0	1
0	0	1
1	1	0

La propagation de l'inconnue bit à bit peut alors s'intuité de la manière suivante :

$$\begin{array}{r} ? \\ \oplus\ 1 \\ \hline = \ ? \end{array} \qquad \begin{array}{r} 1 \\ \oplus\ ? \\ \hline = \ ? \end{array} \qquad \begin{array}{r} ? \\ \oplus\ 0 \\ \hline = \ ? \end{array} \qquad \begin{array}{r} 0 \\ \oplus\ ? \\ \hline = \ ? \end{array} \qquad \begin{array}{r} ? \\ \oplus\ ? \\ \hline = \ ? \end{array}$$

Pour deux tristates $t_1 = (v_1, m_1)$ et $t_2 = (v_2, m_2)$, on définit :

$$v = v_1 \oplus v_2,$$

et on choisit de reprendre le masque commun, ici $\mu = m_1 \vee m_2$ car l'inconnue est toujours propagée. Cette propagation de l'inconnue doit aussi être retrouvé dans la valeur. Pour cela il suffit de mettre à 0 les bits inconnus avec `&&&(~~~`μ`)`. En ∃∀ℕ , l'opération se définit alors ainsi :

```
def xor (t₁ t₂ : Tnum) : Tnum :=
   let val := t₁.v ^^^ t₂.v
   let μ := t₁.m ||| t₂.m
   { v := val &&& (~~~μ), m := μ,
      wf := by
      rw [UInt64.and_assoc],
      have : ~~~μ &&& μ = 0 := by rw [UInt64.and_comm, UInt64.and_not_eq_zero],
      rw [this, UInt64.zero_and]
   }
```

Ici, la fonction `^^^` représente le XOR sur `UInt64`.

Ces définitions garantissent que pour tout opérateur binaire $o \in \{\wedge, \vee, \oplus\}$ défini sur *UInt64*, l'opération correspondante sur `Tnum` envoie un couple `(v,m)` vérifiant bien la condition `v &&& m = 0`. En d'autres termes, les opérations respectent la structure interne d'un tristate.

3.3.3 Opérations arithmétiques

Addition : Une fois les opérations binaires définis, on veut aussi pouvoir définir les opérations $+$ et $\times$ pour les tristates ce qui est beaucoup moins intuitif. On voudrait définir l'addition bit à bit de la manière suivante

$$\begin{array}{r} 0 \\ +\ 0 \\ \hline =\ 0 \end{array} \qquad \begin{array}{r} 1 \\ +\ 0 \\ \hline =\ 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 1 \\ \hline =\ 10 \end{array} \qquad \begin{array}{r} ? \\ +\ 0 \\ \hline =\ ? \end{array} \qquad \begin{array}{r} ? \\ +\ 1 \\ \hline =\ ?? \end{array} \qquad \begin{array}{r} ? \\ +\ ? \\ \hline =\ ?? \end{array}$$

On peut alors faire des calculs posé :

$$\begin{array}{r} 1?01 \\ +\ 1??0 \\ \hline =\ 1???1 \end{array}$$

Vérifions ce que donne le résultat par rapport à l'addition sur les entiers que l'on connait :

$$1?01 = \{9, 13\}$$

$$1??0 = \{8, 10, 12, 14\}$$

$$1???1 = \{17, 19, 21, 23, 25, 27, 29, 31\}$$

Et en sommant les éléments des ensembles 2 à 2 :

$$\{9, 13\} + \{8, 10, 12, 14\} = \{17, 19, 21, 23, 25, 27\}$$

Le résultat $1???1$ est en fait le plus petit tristate au sens du nombre de valeur contenant $\{17, 19, 21, 23, 25, 27\}$.On peut désormais comparer notre approche intuitive avec l'implémentation effective dans ∃∀N . Voici la définition de l'addition entre deux `Tnum` dans ∃∀N :

```
def add (t₁ t₂ : Tnum) : Tnum :=
   let sv := t₁.v + t₂.v
   let sm := t₁.m + t₂.m
   let S := sv + sm
   let q := S ^^^ sv
   let h := q ||| t₁.m ||| t₂.m
   { v := sv &&& (~~~h), m := h,
      wf := by
      rw [UInt64.and_assoc]
      have : ~~~h &&& h = 0 := by rw [UInt64.and_comm, UInt64.and_not_eq_zero]
      rw [this, UInt64.and_zero]
   }

instance : Add Tnum := ⟨ add ⟩
```

La variable `sv` représente la somme des valeurs connues $\texttt{t}_1\texttt{.v} + \texttt{t}_2\texttt{.v}$: c'est ce que l'on obtiendrait si tous les bits inconnus valaient zéro. C'est la borne inférieur de la somme de deux valeurs concrètes respectivement dans $\texttt{t}_1$ et $\texttt{t}_2$. La variable `sm` est la somme des masques $\texttt{t}_1\texttt{.m} + \texttt{t}_2\texttt{.m}$, qui surestime l'incertitude potentielle introduite par les bits inconnus. On forme alors `S = sv + sm`, la borne supérieure de la somme (dans le pire des cas tous les bits inconnus vaudraient 1). En comparant `S` et `sv` via un XOR (`q := S ^^^ sv`), on obtient exactement les bits qui peuvent varier entre la borne inférieure et la borne supérieure. Ce sont les bits potentiellement affectés par les incertitudes dans les opérandes. On complète ensuite ce masque en y ajoutant les incertitudes déjà présentes dans $\texttt{t}_1$ et $\texttt{t}_2$: `h := q |||` $\texttt{t}_1\texttt{.m}$ `|||` $\texttt{t}_2\texttt{.m}$. Finalement, la valeur certaine est extraite en forçant à zéro tous les bits marqués comme incertains : `v := sv &&& (~~~h)`. On obtient ainsi un nouveau `Tnum` avec une valeur `v` garantie sans conflit avec le masque `m`, où le masque `m = h` contient exactement les bits susceptibles de varier dans la somme réelle.

Cette définition réalise exactement ce que l'on cherchait : produire un `Tnum` représentant toutes les sommes possibles de deux `Tnum` tout en étant le plus précis possible, c'est-à-dire avec le masque minimal assurant la couverture de l'ensemble des résultats. Autrement dit, si t_1 représente un ensemble $T_1 \subset \mathbb{N}$ et t_2 un ensemble T_2, alors `add` $\texttt{t}_1$ $\texttt{t}_2$ représente l'ensemble $T_1 + T_2 = \{x + y \mid x \in T_1, y \in T_2\}$, encodé sous forme d'un `Tnum` avec un masque aussi restreint que possible. Cette définition est donc à la fois correcte (le résultat contient bien toutes les sommes possibles) et optimale (le masque ne contient aucun bit inutilement incertain). C'est une généralisation élégante de l'addition binaire, où les incertitudes sont propagées de manière contrôlée.

Multiplication : L'addition n'était déjà pas triviale à généraliser avec des bits inconnus, mais la multiplication est encore un cran au-dessus. On ne peut plus simplement "propager" l'incertitude linéairement, car chaque bit incertain peut impacter tous les bits de poids supérieurs lors de la multiplication.

La définition suivante, écrite en ∃∀N , réalise cette opération :

```
def mul (t₁ t₂ : Tnum) : Tnum :=
   let val := t₁.v * t₂.v
   let mask := { v := 0, m := 0 ,wf := by admit}
   let rec loop (t₁ t₂ mask : Tnum) : Tnum :=
      if t₁.v = 0 ∧ t₁.m = 0 then mask
      else
      let mask :=
         if t₁.v &&& 1 = 1 then mask + { v := 0, m := t₂.m , wf := by rw [UInt64.zero_and]}
         else if t₁.m &&& 1 = 1 then mask + { v := 0, m := t₂.v ||| t₂.m, wf := by rw [
    UInt64.zero_and]}
         else mask
      loop (rshift t₁ 1) (lshift t₂ 1) mask
   termination_by (t₁.v ||| t₁.m)
   decreasing_by
      all_goals simp_wf
      · dsimp [rshift, UInt64.toNat]
      admit
   { v := val, m := 0 , wf := by rw [UInt64.and_zero]} + (loop t₁ t₂ mask)

instance : Mul Tnum := ⟨ mul ⟩
```

Le principe est basé sur la multiplication binaire par décalage (comme celle apprise à l'école primaire, mais en base 2 et avec des inconnues). On commence par calculer la valeur certaine `val := t₁.v * t₂.v`, en supposant que tous les bits inconnus valent 0. Puis, la fonction `loop` balaye $\mathtt{t_1}$ bit à bit, et à chaque étape :

- Si le bit courant de $\mathtt{t_1}$`.v` est 1, on doit ajouter l'incertitude venant de $\mathtt{t_2}$`.m` : c'est-à-dire, on ne connaît pas certains bits dans $\mathtt{t_2}$, donc leur contribution est incertaine.
- Si le bit courant de $\mathtt{t_1}$`.m` est 1 (autrement dit, on ne sait pas si c'est 0 ou 1), alors on ajoute toute l'incertitude venant de $\mathtt{t_2}$, car ce bit peut faire apparaître ou non une copie entière de $\mathtt{t_2}$.

On ajoute ces incertitudes au masque courant via une addition, puis on décale $\mathtt{t_1}$ vers la droite (division par 2), et $\mathtt{t_2}$ vers la gauche (multiplication par 2), en reproduisant l'algorithme classique. Une fois la boucle terminée, on a un masque qui encode toute l'incertitude accumulée par les bits incertains dans les deux opérandes. On l'ajoute à la valeur certaine obtenue au début via la fonction `add` définit précédemment, instanciée par le symbole `+`. Cette approche garantit que la valeur du résultat représente bien le produit minimal (en supposant tous les bits inconnus à 0), que le masque couvre tous les cas où les incertitudes dans les bits de $\mathtt{t_1}$ ou $\mathtt{t_2}$ pourraient changer le résultat et que le `Tnum` obtenu encode toutes les valeurs possibles du produit entre les deux `Tnum`. Ainsi, `mul` $\mathtt{t_1}$ $\mathtt{t_2}$ représente correctement l'ensemble $\{x \cdot y \mid x \in t_1, y \in t_2\}$. Certes, cette méthode peut surapproximer l'incertitude (on ne cherche pas le masque minimal comme pour l'addition), mais elle reste correcte, simple à implémenter, et fiable dans un cadre formel comme ∃∀N . L'optimisation du masque pourrait faire l'objet d'une amélioration ultérieure, mais la priorité ici est la justesse.

3.4 Les intervalles d'entiers 64 bits et le type `I`

3.4.1 Définition

On définit le type `I` qui représente les intervalles de `UInt64` en utilisant le type `Interval` de la bibliothèque `MathLib`. Celui-ci prend en argument un type α muni d'un ordre partiel, et associe à ce type un intervalle muni de quelques propriétés utiles. Un intervalle i est donc un couple $(a, b) \in \mathit{UInt64} \times \mathit{UInt64}$ tel que $a \leq b$, ou bien l'intervalle vide $\emptyset$. Contrairement au type `Tnum` qui n'a pas d'élément vide, `I` possède bien une telle valeur. Cela va impliquer, dans les preuves, de faire souvent des disjonctions de cas selon que l'intervalle est vide ou non. On dit qu'une valeur concrète $u \in \mathit{UInt64}$ appartient à l'intervalle non vide $i = (a, b)$ si $a \leq u \leq b$.

3.4.2 Opérations arithmétiques

Commençons par l'addition d'intervalles. Une idée naturelle consiste à définir :

$$\begin{array}{cccc} + & : & I \times I & \to & I \\ & & (i_1 = (a_1, b_1), i_2 = (a_2, b_2)) & \mapsto & (a_1 + a_2, b_1 + b_2) \end{array}$$

Cependant, deux subtilités importantes doivent être prises en compte :

1. L'écriture $i = (a, b)$ n'a pas de sens lorsque i est vide. Il faut donc traiter ce cas séparément.
2. Les additions sur `UInt64` sont modulaires (i.e. en modulo 2^{64}). Cela peut produire des résultats incohérents lorsqu'un dépassement a lieu. Par exemple :

$$[0, 2^{64} - 1] + [1, 2] = [1, 1]$$

car $(2^{64} - 1) + 2 \equiv 1 \mod 2^{64}$.

Pour éviter ces incohérences, on définit l'addition d'intervalles comme suit :

$$i_1 + i_2 = \begin{cases} \emptyset & \text{si } i_1 = \emptyset \text{ ou } i_2 = \emptyset, \\ [0, 2^{64} - 1] & \text{si } a_1 + a_2 \geq 2^{64} \text{ ou } b_1 + b_2 \geq 2^{64}, \\ [a_1 + a_2, b_1 + b_2] & \text{sinon.} \end{cases}$$

Autrement dit, on renvoie l'intervalle maximal dès qu'un dépassement est possible. Cet intervalle $[0, 2^{64} - 1]$ ne contient aucune information utile (il contient tous les `UInt64`) mais reste un intervalle correct. On peut alors écrire cette addition dans Lean :

```
def add (i₁ i₂ : I) : I :=
  match i₁ with
  | none => none
  | some i₁ =>
    match i₂ with
    | none => none
    | some i₂ =>
      let h := i₁.snd.toNat + i₂.snd.toNat ≤ UInt64.size - 1
      let j := i₁.fst.toNat + i₂.fst.toNat ≤ UInt64.size - 1
      if h && j then
        some {
        fst := i₁.fst + i₂.fst,
        snd := i₁.snd + i₂.snd,
        fst_le_snd := by
        apply UInt64.add_le_add
        · exact i₁.fst_le_snd
        · exact i₂.fst_le_snd
        admit
        }
      else
        some {
        fst := 0,
        snd := (UInt64.size - 1).toUInt64,
        fst_le_snd := by admit
        }

instance : Add I := ⟨ add ⟩
```

Les preuves que les intervalles soient bien formés restent à faire. La multiplication se fait de manière totalement analogue, avec les mêmes principes :

$$i_1 \cdot i_2 = \begin{cases} \emptyset & \text{si } i_1 = \emptyset \text{ ou } i_2 = \emptyset, \\ [0, 2^{64} - 1] & \text{si } a_1 \cdot a_2 \geq 2^{64} \text{ ou } b_1 \cdot b_2 \geq 2^{64}, \\ [a_1 \cdot a_2, b_1 \cdot b_2] & \text{sinon.} \end{cases}$$

Ce qui donne dans Lean :

```
def mul (i₁ i₂ : I) : I :=
  match i₁ with
  | none => none
  | some i₁ =>
    match i₂ with
    | none => none
    | some i₂ =>
      let h := i₁.snd.toNat * i₂.snd.toNat ≤ UInt64.size - 1
      let j := i₁.fst.toNat * i₂.fst.toNat ≤ UInt64.size - 1
      if h && j then
        some {
        fst := i₁.fst * i₂.fst,
        snd := i₁.snd * i₂.snd,
        fst_le_snd := by admit
        }
      else
        some {
        fst := 0,
        snd := (UInt64.size - 1).toUInt64,
        fst_le_snd := by admit
        }

instance : Mul I := ⟨ mul ⟩
```

Là encore les preuves du fait que les intervalles soient bien formé reste à faire. On veut ici manipuler des *intervalles de valeurs possibles* pour les entiers 64 bits, tout en conservant des bornes bien définies et en évitant les incohérences dues aux débordements. À cet effet, notre définition respecte trois principes fondamentaux :

- **Propagation du vide** : si l'un des intervalles est vide, le résultat est également vide. C'est une règle naturelle de l'arithmétique des ensembles : $\emptyset + I = \emptyset$ et $\emptyset \cdot I = \emptyset$.
- **Sur-approximation en cas de dépassement** : lorsqu'une addition ou une multiplication pourrait dépasser $2^{64} - 1$, on préfère renvoyer l'intervalle maximal $[0, 2^{64} - 1]$ qui contient toutes les valeurs possibles d'un `UInt64`. Cela garantit que même si l'on ne sait pas ce qu'il se passe exactement, on est sûr de ne pas rater de valeurs possibles. C'est une forme de précaution calculée.
- **Forme valide d'intervalle** : les constructions de la forme `{ fst := ..., snd := ..., fst_le_snd := ... }` garantissent que les bornes de l'intervalle sont bien dans le bon ordre, et ∃∀N vérifie cela via les preuves demandées dans le champ `fst_le_snd`.

Dans l'implémentation ∃∀N de l'addition, on vérifie d'abord si les deux extrémités (inférieure et supérieure) de l'intervalle résultant ne provoquent pas de débordement. Pour cela, on passe par une conversion en `Nat` :

```
let h := i₁.snd.toNat + i₂.snd.toNat ≤ UInt64.size - 1
```

où `UInt64.size - 1` représente la borne supérieure de `UInt64`. Si le test est réussi, on renvoie un nouvel intervalle avec les bornes sommées, ce qui est la version classique et intuitive de l'addition d'intervalles. Sinon, on renvoie l'intervalle maximal. La multiplication suit le même schéma : on teste les bornes extrêmes $a_1 \cdot a_2$ et $b_1 \cdot b_2$, et on applique la même logique de précaution. Bien que certaines preuves soient encore laissées en `admit` (ce qui est courant en cours de formalisation), leur contenu est raisonnablement évident : on veut prouver que la somme (ou le produit) de deux bornes inférieures reste bien inférieure ou égale à la somme (ou le produit) des bornes supérieures, ce qui est garanti par la monotonie de `+` et `*` sur les `UInt64`, tant que les dépassements sont évités. Ce comportement reste pessimiste : en cas de doute sur un dépassement, on renvoie l'intervalle total. Mais cela permet de préserver l'invariant : si $x \in i_1$ et $y \in i_2$ alors $x + y \in i_1 + i_2$ et $x \cdot y \in i_1 \cdot i_2$, ce qui est le but recherché. On est donc dans une logique d'inclusion garantie.

3.4.3 Opérations croisées entre `Tnum` et `I`

On a vu qu'une valeur abstraite est constituée à la fois d'un `Tnum` et d'un `I`. Une idée naturelle, plutôt que de définir des opérations séparément sur les `Tnum` d'un côté et sur les `I` de l'autre, est de considérer des opérations *croisées* qui utilisent les deux simultanément. En effet, notre objectif final est de manipuler des valeurs abstraites complètes. Dans la majorité des cas, il est préférable d'appliquer les opérations sur chaque composant séparément : on obtient alors des résultats explicites, parfois même

exacts. Cependant, lorsque ce n'est pas possible, notamment pour certaines opérations binaires difficiles à exprimer directement sur les intervalles, et il peut être pertinent d'approximer le résultat à l'aide des `Tnum` associés. Prenons par exemple l'opération $\vee$ (OU binaire) sur des intervalles. Il n'est pas trivial de la définir uniquement à partir des bornes. On peut cependant utiliser la propriété suivante :

$$\forall x, y, a \in \mathit{UInt64},\ x \geq a \Rightarrow x \vee y \geq a \vee y$$

Intuitivement, cela reflète le fait que l'opérateur $\vee$ ne peut qu'ajouter des bits à 1, mais jamais en retirer. Par conséquent :

$$\forall x \in [a_1, b_1],\ \forall y \in [a_2, b_2],\ x \vee y \geq a_1 \vee y \geq a_1 \vee a_2$$

Cela nous donne une borne inférieure pour l'intervalle $[a_1, b_1] \vee [a_2, b_2]$, à savoir $a_1 \vee a_2$. Pour la borne supérieure, on s'appuie sur les `Tnum`. Si $\texttt{t}_1$ et $\texttt{t}_2$ sont deux `Tnum`, alors la valeur maximale que peut prendre $\texttt{t}_1$ `|||` $\texttt{t}_2$ `= (v,m)` est donnée par :

```
v ||| m
```

Soient $(A_1 = T, i_1, t_1)$ et $(A_2 = T, i_2, t_2)$ deux valeurs abstraites initialisées. On note $t = t_1 \vee t_2$. On peut alors définir l'intervalle résultant de leur disjonction binaire comme suit, avec $t = (v, m)$:

$$i_1 \vee i_2 = \begin{cases} \emptyset & \text{si } i_1 = \emptyset \text{ ou } i_2 = \emptyset, \\ [a_1 \vee a_2,\ v \vee m] & \text{sinon.} \end{cases}$$

Cette définition combine une borne inférieure déterministe (grâce à la monotonie de $\vee$) et une borne supérieure surestimée via le `Tnum`, ce qui reste valide dans une interprétation abstraite. On écrit alors dans ∃∀N l'opération `or` qui prend une `Tnum t`, qui représente $\texttt{t}_1$`|||`$\texttt{t}_2$, et deux `I` :

```
def or (t : Tnum) (i₁ i₂ : I) : I :=
  match i₁ with
  | none => none
  | some i₁ =>
    match i₂ with
    | none => none
    | some i₂ =>
      some {
      fst := i₁.fst ||| i₂.fst,
      snd := t.v ||| t.m
      fst_le_snd := by sorry
      }
```

Dans la même idée, on peut définir l'opération $\wedge$ sur les intervalles, en se basant à la fois sur les bornes et sur les `Tnum` associés.

Rappelons que l'opérateur $\wedge$ (ET binaire) ne peut que faire passer des bits de 1 à 0 : il est monotone *décroissant* au sens où :

$$\forall x, y, b \in \mathit{UInt64},\ x \leq b \Rightarrow x \wedge y \leq b \wedge y$$

Ainsi, pour tout $x \in [a_1, b_1]$ et $y \in [a_2, b_2]$, on a :

$$x \wedge y \leq b_1 \wedge y \leq b_1 \wedge b_2$$

ce qui fournit une borne supérieure sur l'opération $[a_1, b_1] \wedge [a_2, b_2]$. Pour la borne inférieure, on peut réutiliser le `Tnum` calculé `t =` $\texttt{t}_1$ `&&&` $\texttt{t}_2$: sa valeur est une approximation minimale correcte de tous les résultats possibles de $x \wedge y$, puisque tout bit qui vaut 0 dans l'un des deux opérandes y restera.

On peut donc poser pour t = (v,m) :

$$i_1 \wedge i_2 = \begin{cases} \emptyset & \text{si } i_1 = \emptyset \text{ ou } i_2 = \emptyset, \\ [v,\ b_1 \wedge b_2] & \text{sinon.} \end{cases}$$

Et l'implémentation en ∃∀N peut se faire ainsi :

```
def and (t : Tnum) (i₁ i₂ : I) : I :=
  match i₁ with
  | none => none
```

```
| some i₁ =>
  match i₂ with
  | none => none
  | some i₂ =>
    some {
    fst := t.v,
    snd := i₁.snd &&& i₂.snd,
    fst_le_snd := by admit
    }
```

3.5 Sémantique abstraite

Soit P un programme de longueur n et S_i un état abstrait initial. On veut définir la propagation des états abstraits le long du flot de contrôle du programme P, en fonction de la position courante dans le programme (le compteur de programme, noté pc), et de l'historique de propagation.

3.5.1 Ensemble des états par point de programme

Pour chaque position k dans le programme (c'est-à-dire chaque instruction), on définit un ensemble Σ_k d'états abstraits qui atteignent l'instruction k. On distingue deux sous-ensembles :

$$\Sigma_k = (\Sigma_k^*, \overline{\Sigma_k}) \quad \text{où tous les états } s \in \Sigma_k \text{ vérifient } s.pc = k$$

— Σ_k^* est l'ensemble des états déjà propagés depuis k ;
— $\overline{\Sigma_k}$ est l'ensemble des états encore à propager depuis k.

3.5.2 Initialisation

On initialise l'analyse avec un seul état abstrait S_i positionné sur la première instruction ($pc = 1$), et aucun état dans les autres positions du programme :

$$\begin{bmatrix} \Sigma_1 = (\Sigma_1^* = \emptyset, \overline{\Sigma_1} = \{S_i\}), & \Sigma_2 = \emptyset, & \ldots, & \Sigma_n = \emptyset \\ & (1) & & \end{bmatrix}$$

La deuxième ligne représente la file de travail, ici initialement le singleton (1), indiquant que l'instruction à l'indice 1 est en attente de traitement.

3.5.3 Étape de propagation (récurrence)

On suppose maintenant que l'on a une configuration :

$$\begin{bmatrix} (\Sigma_1 = (\Sigma_1^*, \overline{\Sigma_1}), \Sigma_2, \ldots, \Sigma_n) \\ (u_1, u_2, \ldots, u_k) \end{bmatrix}$$

où $(u_1, \ldots, u_k)$ est la file de travail (les positions du programme où l'on doit encore propager les états non traités).

On considère l'instruction à la position u_1, notée $P[u_1]$. En appliquant cette instruction à tous les états de $\overline{\Sigma_{u_1}}$, on obtient un ensemble d'états que l'on regroupe en fonction de la nouvelle valeur de leur compteur de programme :

$$\forall j \in [1, n], \quad I_{u_1}^j = \{s \in P[u_1](\overline{\Sigma_{u_1}}) \mid s.pc = j\}$$

Autrement dit, $I_{u_1}^j$ est l'ensemble des états produits par l'instruction $P[u_1]$ qui poursuivent leur exécution à la position j.

On met alors à jour les ensembles Σ_j de la manière suivante :

$$\Sigma'_j = \Sigma_j \cup I_{u_1}^j \quad \text{pour tout } j \neq u_1$$

$$\Sigma'_{u_1} = \left(\Sigma_{u_1}^* \cup \overline{\Sigma_{u_1}},\ I_{u_1}^{u_1}\right)$$

Enfin, on met à jour la file de travail avec les indices $v_1, \ldots, v_m$ tels que $I_{u_1}^{v_j} \neq \emptyset$, autrement dit tous les points du programme pour lesquels de nouveaux états ont été produits :

$$\begin{bmatrix} (\Sigma'_1, \ldots, \Sigma'_n) \\ (v_1, \ldots, v_m,\ u_2, \ldots, u_k) \end{bmatrix}$$

Ce processus continue jusqu'à épuisement de la file de travail.

3.5.4 Esquisse d'implémentation

Pour mettre en œuvre l'analyse abstraite décrite précédemment, on introduit plusieurs structures de données en ∃∀ℕ qui reflètent notre modélisation mathématique.

Ensembles d'états abstraits par point de contrôle. On définit d'abord le type d'un ensemble d'états abstraits associés à une instruction donnée du programme. Chaque tel ensemble est composé d'un sous-ensemble d'états déjà propagés (`p` pour "propagés) et d'un sous-ensemble d'états en attente de propagation (`np` pour "non propagés"). Un invariant garantit que tous les états contenus dans la structure pointent vers la même position `n` du programme :

```
abbrev StSet := Set AbsState

structure pStSet where
   n : Nat
   p : StSet
   np : StSet
   wf : ∀ s ∈ (p ∪ np), s.pc = n
```

On peut initialiser une telle structure vide via une instance de collection vide :

```
instance : EmptyCollection pStSet :=
   ⟨0, ∅ , ∅ , by intro h; rw [Set.empty_union]; exact (Set.not_mem_empty h).elim⟩
```

Table des états abstraits. La table globale d'analyse, appelée `StTable`, associe à chaque instruction du programme un tel `pStSet`. Un invariant de bonne formation garantit que cette table a la même longueur que le programme analysé :

```
structure StTable (p : Program) where
   stTable : List pStSet
   wf : stTable.length = p.length
```

On peut construire une table vide à partir de la taille du programme :

```
def empty_StTable (n : Nat) : List pStSet := List.replicate n ∅
```

Puis initialiser cette table avec un état initial s_i placé au début du programme :

```
def init_StTable (p : Program) (sᵢ : pStSet) : StTable p :=
   if p.length ≥ 1 then
   let l : List pStSet := empty_StTable (p.length - 1)
   ⟨sᵢ :: l , by admit⟩
   else
   ⟨ [sᵢ], by admit ⟩
```

File de travail. On définit ensuite une structure combinant la table d'états abstraits avec une file de travail `pcTable`, modélisée comme une liste d'indices pointant vers les instructions du programme à traiter :

```
abbrev Pindex (p : Program) := Fin p.length
abbrev Lpindex (p : Program) := List (Pindex p)

structure Table (p : Program) where
   stTable : StTable p
   pcTable : Lpindex p
```

Moteur de propagation. La fonction suivante esquisse une étape de propagation abstraite. Elle extrait l'indice en tête de la file de travail, récupère l'instruction correspondante, et prépare l'application de sa sémantique abstraite. Le cœur de cette fonction reste à compléter :

```
def new_Table (p : Program) (t : Table p) : Table p :=
   match t.pcTable with
   | [] => t  -- plus rien à propager
   | u₁ :: rpc =>
```

```
have : u₁ < p.length := by sorry
let I₁ : Statement := p[u₁]
sorry
```

Cette architecture permet de formaliser proprement la propagation des états abstraits dans un programme, en séparant les responsabilités :

— `pStSet` gère les états par instruction avec distinction entre propagés et non propagés,
— `StTable` contient la mémoire globale de l'analyse abstraite,
— `Table` organise l'ensemble du processus avec une file de propagation.

Le développement futur consistera à compléter la définition de `new_Table`, en implémentant la sémantique abstraite des instructions du langage, ainsi que la mise à jour des ensembles d'états et de la file de travail.

4 Conclusion

4.1 Travail accompli

Le travail présenté dans ce rapport s'inscrit dans une perspective ambitieuse : la construction d'un compilateur certifié pour eBPF, en s'appuyant sur l'assistant à la preuve Lean. Dans ce contexte, plusieurs étapes fondamentales ont été réalisées. Dans un premier temps, une formalisation rigoureuse d'un sous-ensemble du langage eBPF a été proposée. Cette formalisation inclut la définition syntaxique des instructions élémentaires (affectations, opérations arithmétiques et binaires, sauts, appels de fonction, sortie), ainsi qu'une sémantique opérationnelle concrète décrivant l'évolution des états d'un programme lors de son exécution. La deuxième partie du travail a porté sur l'introduction d'une sémantique abstraite, dans le but de rendre possible une analyse statique formelle. Deux domaines abstraits ont été définis et intégrés : les `Tnum`, qui modélisent l'incertitude bit à bit, et les `I`, qui encadrent les valeurs possibles d'un registre. Des opérateurs arithmétiques et binaires ont été soigneusement définis sur ces deux domaines, en prenant en compte les spécificités du type `UInt64`, notamment les effets de débordement. Enfin, une ébauche d'implémentation a été proposée pour structurer les ensembles d'états abstraits par point de programme, via une table d'état à deux composantes (états propagés et non propagés). Cette structure constitue la première pierre d'un moteur de propagation abstraite itératif, analogue à celui du vérificateur eBPF du noyau Linux. Ce travail jette ainsi les bases d'un analyseur abstrait formellement vérifié, et constitue une étape essentielle vers un compilateur certifié pour eBPF.

4.2 Perspectives

L'objectif à terme serait de garantir formellement que tout programme produit par ce compilateur est sûr, termine correctement, et respecte les contraintes imposées par le noyau Linux, le tout de manière vérifiée dans Lean. Cependant, il reste encore beaucoup à faire. La formalisation proposée ici ne couvre qu'un sous-ensemble restreint d'instructions eBPF. Il faudra l'étendre progressivement pour intégrer l'ensemble du langage, notamment les accès mémoire, les pointeurs, la pile, et les interactions avec les structures de données spécifiques à eBPF (maps, helpers, etc.). Ces éléments introduisent des aspects complexes, notamment en ce qui concerne la mémoire, la validité des pointeurs par exemple. D'un point de vue sémantique, la propagation d'états abstraits reste à implémenter dans son intégralité. Le moteur d'exécution abstraite n'est pour l'instant qu'ébauché, et son fonctionnement doit encore être précisé, puis intégré à un cadre général de vérification. À terme, un objectif essentiel sera de prouver la correction d'un compilateur eBPF écrit en Lean, c'est-à-dire de garantir que le programme machine généré par ce compilateur est conforme à la sémantique du programme source, tout en étant accepté par le vérificateur du noyau. Ainsi, ce travail constitue un point de départ nécessaire, mais encore très partiel, d'un projet à long terme.

4.3 Expérience personnelle

Ce stage a été une expérience particulièrement formatrice, aussi bien sur le plan technique que personnel. Dès les premières semaines, j'ai abordé le projet avec une grande motivation. J'ai rapidement contacté mes encadrants, commencé à apprendre Lean de manière autonome pendant les vacances, et pris l'habitude de me rendre chaque vendredi au laboratoire. Cette autonomie initiale a marqué le ton du stage : avancer seul, expérimenter, me confronter directement au sujet. Très vite, cependant, j'ai mesuré la complexité du sujet. Les premières difficultés sont apparues rapidement, et il m'arrivait de travailler

longuement sans réussir à faire avancer le projet de manière concrète ou bien même ne pas comprendre une notion pendant des semaines. Cette stagnation a engendré du stress, notamment parce que je tenais à ne pas décevoir mes encadrants. Je savais qu'un des membres de l'équipe avait lui-même suivi un parcours recherche dans ce domaine, avec succès. Cela a pu renforcer l'impression de ne pas être à la hauteur des attentes.

Les incompréhensions techniques, la lenteur des progrès et le sentiment d'isolement ont parfois été frustrants, surtout lorsqu'on compare l'intensité de l'effort investi aux résultats visibles. Mais ce stress et cette frustration m'ont aussi appris à gérer mes attentes, à rester patient, et à construire une dynamique de travail autonome, même sans retour immédiat. Pour maintenir le cap, j'ai misé sur la régularité et la discipline : continuer à venir au laboratoire, organiser mes sessions de travail, chercher à comprendre progressivement plutôt que chercher à tout résoudre d'un coup. J'ai appris à faire avec l'incertitude, à repartir de zéro quand une piste ne menait nulle part, et à documenter mes avancées même lorsqu'elles semblaient minimes.

Sur le plan technique, le principal apport du stage est sans doute ma découverte du langage LEAN et plus généralement des méthodes formelles. Travailler avec un assistant à la preuve m'a forcé à être rigoureux et à formaliser pas à pas ce que l'on écrit souvent en quelques lignes dans un papier. J'ai également appris ce que signifie formaliser un langage de programmation : construire sa syntaxe, en définir la sémantique, et poser les bases d'une exécution abstraite. Ce travail m'a permis de mettre en place une vision claire de la manière dont on passe du code à sa signification mathématique. Bien sûr, tout n'a pas été optimal : certains efforts se sont révélés inutiles à terme, comme l'apprentissage des méthodes Python pour exécuter des programmes eBPF, ou la redéfinition manuelle d'objets déjà présents dans `MathLib` comme les intervalles. Mais ces détours font aussi partie de l'apprentissage.

Je remercie sincèrement mes tuteurs Ghilain BERGERON, Stephan MERZ et Vincent TRÉLAT de l'équipe VERIDIS du LORIA pour leur accueil, leur disponibilité, leur exigence mais aussi leur bienveillance tout au long du projet. Les échanges réguliers, les conseils reçus et l'ambiance de travail m'ont permis d'apprendre à grande vitesse et de me projeter dans le monde de la recherche. Je remercie Yannick PRIVAT pour m'avoir offert cette opportunité et pour avoir su me proposer un parcours de recherche en parfaite résonance avec ma curiosité pour les méthodes formelles.

Enfin, malgré les obstacles rencontrés, ce stage m'a conforté dans mon intérêt pour la recherche. J'ai pu entrevoir ce que signifie faire progresser un sujet original, avec tout ce que cela implique de doute, de rigueur et de créativité. Aujourd'hui, j'envisage avec enthousiasme de poursuivre dans cette voie.

Annexes

A Codes Lean

```
/--Function that takes a function g from Register to Val, a Register α and a Val v and return
    a function f so that f(α) = v and f(r!=α)=g(r)-/
def update(g : (Register → Val)) (α : Register) (v : Val) : Register → Val :=
   fun r : Register => if r = α then v else g r

/--Function that takes an Argument src, a Register dst and a State s and return a new State
    where pc+=1 and dst = dst + src-/
def add(dst : Register)(src : Argument)(s : State) : State :=
   match src with
   |Argument.imm v_src => {pc := s.pc + 1, reg := update s.reg dst (s.reg dst + v_src)}
   |Argument.reg r_src => {pc := s.pc + 1, reg := update s.reg dst (s.reg dst + s.reg r_src)}

/--Function that takes an Argument src, a Register dst ans a State s and return a new State
    where pc+=1 and dst = dst or src-/
def or_(dst : Register)(src : Argument)(s : State) : State :=
   match src with
   |Argument.imm v_src => {pc := s.pc + 1, reg := update s.reg dst (s.reg dst ||| v_src)}
   |Argument.reg r_src => {pc := s.pc + 1, reg := update s.reg dst (s.reg dst ||| s.reg r_src)}

/--Function that takes an Argument src, a Register dst ans a State s and return a new State
    with pc=s.pc+1 and dst = src-/
def mov (dst : Register)(src : Argument)(s : State) : State :=
   match src with
   |Argument.imm v_src => {pc := s.pc + 1, reg := update s.reg dst (v_src)}
   |Argument.reg r_src => {pc := s.pc + 1, reg := update s.reg dst (s.reg r_src)}

/--Function that takes an Nat offset and a State s and return the State with pc = pc + offset-/
def ja (offset: Int) (s : State) : State :=
   {s with pc := s.pc+offset}

/--Function that takes two Register src and dst and an offset and a State s and return the
    State with pc = pc + offset if dst = src and pc = pc + 1 otherwise-/
def jeq(dst : Register)(src : Argument)(offset: Int)(s : State) : State :=
   match src with
   |Argument.imm v_src =>
   if s.reg dst = v_src then {s with pc := s.pc + offset} else {s with pc := s.pc + 1}
   |Argument.reg r_src =>
   if s.reg dst = s.reg r_src then {s with pc := s.pc + offset} else {s with pc := s.pc + 1}
```

Code 1 – Fonctions de la sémantique concrète

```
theorem toNat_add_le_umax (h : a.toNat + b.toNat ≤ UInt64.size-1) : a.toNat + b.toNat = (a +
    b).toNat := by admit

theorem toNat_ge_add : (a + b).toNat ≤ a.toNat + b.toNat := by admit
```

Code 2 – Lemmes intermédiares sur le comportement de `.toNat` avec l'addition (admits)

```
theorem max_toNat_eq : (max a b).toNat = max a.toNat b.toNat := by
   change (if a ≤ b then b else a).toNat = if a.toNat ≤ b.toNat then b.toNat else a.toNat
   split <;> split <;> trivial

theorem le_max_left : a ≤ max a b := by
   change  a.toNat ≤ (max a b).toNat
   rw [UInt64.max_toNat_eq]
   apply Nat.le_max_left
```

Code 3 – Propriété $a \leq max(a, b)$

Références

[1] *Theorem Proving in Lean 4. (s. d.).* https://leanprover.github.io/theorem_proving_in_lean4/

[2] *What is eBPF ? An Introduction and Deep Dive into the eBPF Technology.* (s. d.). https://ebpf.io/fr-fr/what-is-ebpf/

[3] *eBPF Instruction Set — The Linux Kernel documentation.* (s. d.). https://www.kernel.org/doc/html/v5.19/bpf/instruction-set.html

[4] Torvalds. (s. d.). *linux/kernel/bpf at master · torvalds/linux.* GitHub. https://github.com/torvalds/linux/tree/master/kernel/bpf

[5] Cousot, P. (2001). Abstract Interpretation Based Formal Methods and Future Challenges. *Lecture notes in computer science* (p. 138-156). https://doi.org/10.1007/3-540-44577-3_10

[6] Vishwanathan, H., Shachnai, M., Narayana, S., & Nagarakatte, S. (2022b). Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers. *Rutgers Department Of Computer Science Technical Report*, 254-265. https://doi.org/10.1109/cgo53902.2022.9741267